# Namespace CJson

## Classes

CJson<Type>

CJson entrypoint class.

Object must be created for any CJson file or content for deserializing.

Only class object to JSON string conversion can be done without object creation.

Path

For file path related operations, use this class object.

# Class CJson<Type>

Namespace: CJson

Assembly: CJson.dll

CJson entrypoint class.
Object must be created for any CJson file or content for deserializing.
Only class object to JSON string conversion can be done without object creation.

```
public sealed class CJson<Type> : Decode
```

## Type Parameters

`Type`

Class Type should be passed. For more info, see official docs

**Inheritance**

object ← Base ← Is ← Json ← Decode ← CJson<Type>

**Inherited Members**

Decode.DecodeKeywords , Decode.InjectData(string, dynamic) , Json.ParseValue(string) , Json.Parse(string) , Is.MatchAndConfirm , Base.Read(string) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## CJson(Path)

Constructor for CJSON/ JSON file path
CJson/JSON file path to be passed with CJson.Path class constructor.
For loading CJSON/JSON raw content, please call CJSON(string content) contructor

```
public CJson(Path filePath)
```

## Parameters

`filePath` Path

CJson.Path class constructor

## CJson(string)

Constructor for raw CJSON/ JSON content
CJson content to be passed.
For loading CJSON/ JSON file, please create constructor with CJson.Path class For import statements, please specify absolute path to the file to be imported.

```
public CJson(string content)
```

### Parameters

`content` string

CJson content in string format

# Methods

## Deserialize()

Call this method to deserialize CJSON content to ClassType.
If any error in format of the class hierarchy exists, this function will return a null valued object to the provided ClassType

```
public Type? Deserialize()
```

### Returns

Type

Deserialized class object of Type. Null if Type and CJSON content mismatches

## DeserializeAsString()

```
public string DeserializeAsString()
```

### Returns

string

## Inject(Dictionary<string, dynamic>)

Call this function to inject value which expects a dynamic injection.
You can use this function for bulk injection.

For more info, check this link

```
public Type? Inject(Dictionary<string, dynamic> injectingValues)
```

### Parameters

`injectingValues` Dictionary<string, dynamic>

Dictionary whose key is location and value is data to be inject

### Returns

Type

## Inject(string, dynamic)

Call this function to inject value which expects a dynamic injection.
**Please note, inject doesnt work on inject at jpath, or inject at any json key.**

For more info, check this link

```
public Type? Inject(string key, dynamic value)
```

### Parameters

`key` string

Where you want to inject

`value` dynamic

Value which you want to inject at key

### Returns

Type

Deserialized class object of Type. Null if Type and CJSON content mismatches

## ToString(dynamic)

Use this function to convert a class object to a JSON string. No CJSON operation can be performed after this function.

```
public static string ToString(dynamic value)
```

### Parameters

`value` dynamic

class object

### Returns

string

# Class Path

Namespace: CJson

Assembly: CJson.dll

For file path related operations, use this class object.

```
public class Path
```

**Inheritance**

object ← Path

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

## Constructors

### Path(params string[])

Constructor for object creation.
Please provide absolute path for this operation.

```
public Path(params string[] filePath)
```

#### Parameters

`filePath` string[]

## Properties

### ToString

Get the string value of the path

```
public string ToString { get; }
```

### Property Value

string

## Methods

### IsAbsolutePath(string)

Returns true if the path is absolute, else false.
This method is not tested in IOS. For bug report, refer to the github

```
public static bool IsAbsolutePath(string filePath)
```

#### Parameters

`filePath` string

Test path

#### Returns

bool

true if path is absolute, else false in boolean

# Namespace CJson.Exceptions

## Classes

AbsolutePathConstraintError

IllegalImportStatement

UnrecognizedTypeException

UnsupportedFileExtension

# Class AbsolutePathConstraintError

Namespace: CJson.Exceptions

Assembly: CJson.dll

```
public class AbsolutePathConstraintError : Exception, ISerializable
```

### Inheritance

object ← Exception ← AbsolutePathConstraintError

### Implements

ISerializable

### Inherited Members

Exception.GetBaseException() , Exception.GetObjectData(SerializationInfo, StreamingContext) , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

## Constructors

### AbsolutePathConstraintError()

```
public AbsolutePathConstraintError()
```

# Class IllegalImportStatement

Namespace: CJson.Exceptions

Assembly: CJson.dll

```
public class IllegalImportStatement : Exception, ISerializable
```

### Inheritance

object ← Exception ← IllegalImportStatement

### Implements

ISerializable

### Inherited Members

Exception.GetBaseException() , Exception.GetObjectData(SerializationInfo, StreamingContext) , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

## Constructors

### IllegalImportStatement()

```
public IllegalImportStatement()
```

### IllegalImportStatement(string)

```
public IllegalImportStatement(string message)
```

#### Parameters

`message` string

# Class UnrecognizedTypeException

Namespace: CJson.Exceptions

Assembly: CJson.dll

```
public class UnrecognizedTypeException : Exception, ISerializable
```

### Inheritance

object ← Exception ← UnrecognizedTypeException

### Implements

ISerializable

### Inherited Members

Exception.GetBaseException() , Exception.GetObjectData(SerializationInfo, StreamingContext) , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

## Constructors

### UnrecognizedTypeException()

```
public UnrecognizedTypeException()
```

### UnrecognizedTypeException(string)

```
public UnrecognizedTypeException(string message)
```

#### Parameters

`message` string

# Class UnsupportedFileExtension

Namespace: CJson.Exceptions

Assembly: CJson.dll

```
public class UnsupportedFileExtension : Exception, ISerializable
```

### Inheritance

object ← Exception ← UnsupportedFileExtension

### Implements

ISerializable

### Inherited Members

Exception.GetBaseException() , Exception.GetObjectData(SerializationInfo, StreamingContext) , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

## Constructors

### UnsupportedFileExtension()

```
public UnsupportedFileExtension()
```

### UnsupportedFileExtension(string)

```
public UnsupportedFileExtension(string message)
```

#### Parameters

`message` string

# Namespace CJson.Types

## Classes

ParsedValue

Try

# Class ParsedValue

Namespace: CJson.Types

Assembly: CJson.dll

```
public class ParsedValue
```

**Inheritance**

object ← ParsedValue

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### ParsedValue(object, string)

```
public ParsedValue(object value, string type)
```

#### Parameters

`value` object

`type` string

## Fields

### type

```
public string type
```

#### Field Value

string

# value

```
public dynamic value
```

## Field Value

dynamic

# Class Try

Namespace: CJson.Types

Assembly: CJson.dll

```
public class Try
```

**Inheritance**

object ← Try

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## GetType(dynamic)

```
public static string GetType(dynamic value)
```

### Parameters

`value` dynamic

### Returns

string

## TryBoolean(JValue)

```
public static bool TryBoolean(JValue jValue)
```

### Parameters

`jValue` JValue

### Returns

bool

## TryDouble(JValue)

```
public static bool TryDouble(JValue jValue)
```

### Parameters

`jValue` JValue

### Returns

bool

## TryInt32(JValue)

```
public static bool TryInt32(JValue jValue)
```

### Parameters

`jValue` JValue

### Returns

bool

## TryNull(JValue)

```
public static bool TryNull(JValue jValue)
```

### Parameters

`jValue` JValue

### Returns

bool

## TryString(JValue)

```
public static bool TryString(JValue jValue)
```

### Parameters

`jValue` JValue

### Returns

bool

# Namespace CJson.Utils

## Classes

Base

Decode

Is

Json

# Class Base

Namespace: CJson.Utils

Assembly: CJson.dll

```
public class Base
```

**Inheritance**

object ← Base

**Derived**

Is

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### Base(string)

```
public Base(string content)
```

#### Parameters

`content` string

### Base(string, bool)

```
public Base(string filePath, bool isFilePath)
```

#### Parameters

`filePath` string

`isFilePath` bool

# Fields

## commaSeparatedLines

```
protected string[] commaSeparatedLines
```

### Field Value

string []

## content

```
protected string content
```

### Field Value

string

## filePath

```
protected string filePath
```

### Field Value

string

## isFilePath

```
protected bool isFilePath
```

### Field Value

bool

# Methods

## GetType(dynamic)

```
protected static string GetType(dynamic testVar)
```

### Parameters

`testVar` dynamic

### Returns

string

## ParseJson(string)

```
protected static dynamic ParseJson(string jsonString)
```

### Parameters

`jsonString` string

### Returns

dynamic

## Read(string)

```
public static string Read(string filePath)
```

### Parameters

`filePath` string

## Returns

string

# Class Decode

Namespace: CJson.Utils

Assembly: CJson.dll

```
public class Decode : Json
```

**Inheritance**

object ← Base ← Is ← Json ← Decode

**Derived**

CJson<Type>

**Inherited Members**

Json.json , Json.ParseValue(string) , Json.RedefineJson(string) , Json.Parse(string) , Is.MatchAndConfirm , Is.Scan() , Base.filePath , Base.content , Base.commaSeparatedLines , Base.isFilePath , Base.Read(string) , Base.ParseJson(string) , Base.GetType(dynamic) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### Decode(Path)

```
public Decode(Path filePath)
```

#### Parameters

`filePath` Path

### Decode(string)

```
public Decode(string content)
```

#### Parameters

`content` string

# Fields

## isDeseralized

```
protected bool isDeseralized
```

### Field Value

bool

## matchedPaths

```
protected List<string> matchedPaths
```

### Field Value

List<string>

## runtimeValList

```
protected static List<string> runtimeValList
```

### Field Value

List<string>

# Properties

## DecodeKeywords

Decode root for all keywords

```
public string DecodeKeywords { get; }
```

### Property Value

string

## Methods

### Get()

```
protected string Get()
```

### Returns

string

### GetAsString(dynamic)

```
protected static string GetAsString(dynamic value)
```

### Parameters

`value` dynamic

### Returns

string

### InjectData(string, dynamic)

```
public string InjectData(string key, dynamic value)
```

### Parameters

`key` string

`value` dynamic

### Returns

string

## ReplaceContent(string, dynamic)

```
protected Decode ReplaceContent(string key, dynamic value)
```

### Parameters

`key` string

`value` dynamic

### Returns

Decode

# Class Is

Namespace: CJson.Utils

Assembly: CJson.dll

```
public class Is : Base
```

**Inheritance**

object ← Base ← Is

**Derived**

Json

**Inherited Members**

Base.filePath , Base.content , Base.commaSeparatedLines , Base.isFilePath , Base.Read(string) , Base.ParseJson(string) , Base.GetType(dynamic) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### Is(string)

```
public Is(string content)
```

#### Parameters

`content` string

### Is(string, bool)

```
public Is(string filePath, bool isFilePath)
```

#### Parameters

`filePath` string

`isFilePath` bool

# Fields

## MatchAndConfirm

```
public static Func<string, string, List<string>> MatchAndConfirm
```

### Field Value

Func<string, string, List<string>>

# Methods

## Scan()

```
protected void Scan()
```

# Class Json

Namespace: CJson.Utils

Assembly: CJson.dll

```
public class Json : Is
```

**Inheritance**

object ← Base ← Is ← Json

**Derived**

Decode

**Inherited Members**

Is.MatchAndConfirm , Is.Scan() , Base.filePath , Base.content , Base.commaSeparatedLines , Base.isFilePath , Base.Read(string) , Base.ParseJson(string) , Base.GetType(dynamic) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### Json(Path)

```
public Json(Path filePath)
```

#### Parameters

`filePath` Path

### Json(string)

```
public Json(string content)
```

#### Parameters

`content` string

# Fields

## json

```
protected dynamic json
```

### Field Value

dynamic

# Methods

## Parse(string)

```
public dynamic Parse(string key)
```

### Parameters

`key` string

### Returns

dynamic

## ParseValue(string)

```
public ParsedValue ParseValue(string key)
```

### Parameters

`key` string

### Returns

ParsedValue

## RedefineJson(string)

```
protected void RedefineJson(string content)
```

### Parameters

`content` string

# Namespace CJsonTests

## Classes

Base

# Class Base

Namespace: CJsonTests

Assembly: CJsonTests.dll

```
public class Base
```

**Inheritance**

object ← Base

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Fields

## cjsonFilePath

```
protected readonly Path cjsonFilePath
```

### Field Value

Path

## jsonfilePath

```
protected readonly Path jsonfilePath
```

### Field Value

Path

## pureJsonfilePath

```
protected readonly Path pureJsonfilePath
```

### Field Value

Path

## relativeTargetCjson

```
protected readonly Path relativeTargetCjson
```

### Field Value

Path

## variableInjectionCjson

```
protected readonly Path variableInjectionCjson
```

### Field Value

Path

# Namespace CJsonTests.models

## Classes

CJsonContentTarget

Pure

Questions

RelativeCalls

Source

Target

TargetObj

TargetRelativeCalls

VariableInjection

# Class CJsonContentTarget

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class CJsonContentTarget
```

**Inheritance**

object ← CJsonContentTarget

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### source

```
public Pure source { get; set; }
```

#### Property Value

Pure

### target

```
public TargetObj target { get; set; }
```

#### Property Value

TargetObj

# Class Pure

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class Pure
```

### Inheritance

object ← Pure

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### quiz

```
public Dictionary<string, Dictionary<string, Questions>> quiz { get; set; }
```

#### Property Value

Dictionary<string, Dictionary<string, Questions>>

# Class Questions

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class Questions
```

**Inheritance**

object ← Questions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## answer

```
public string answer { get; set; }
```

### Property Value

string

## options

```
public List<string> options { get; set; }
```

### Property Value

List <string>

## question

```
public string question { get; set; }
```

## Property Value

string

# Class RelativeCalls

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class RelativeCalls
```

### Inheritance

object ← RelativeCalls

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### quiz

```
public Dictionary<string, Dictionary<string, Questions>> quiz { get; set; }
```

#### Property Value

Dictionary<string, Dictionary<string, Questions>>

# Class Source

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class Source
```

**Inheritance**

object ← Source

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### pure

```
public Pure pure { get; set; }
```

#### Property Value

Pure

# Class Target

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class Target
```

**Inheritance**

object ← Target

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### source

```
public Source source { get; set; }
```

#### Property Value

Source

### target

```
public TargetObj target { get; set; }
```

#### Property Value

TargetObj

# Class TargetObj

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class TargetObj
```

**Inheritance**

object ← TargetObj

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### color

```
public string color { get; set; }
```

#### Property Value

string

### colorList

```
public List<string> colorList { get; set; }
```

#### Property Value

List<string>

### digitArrayImport

```
public List<double> digitArrayImport { get; set; }
```

### Property Value

List <double>

## digitCheck

```
public double digitCheck { get; set; }
```

### Property Value

double

## digitImport

```
public double digitImport { get; set; }
```

### Property Value

double

## fruit

```
public string fruit { get; set; }
```

### Property Value

string

## fruitType

```
public string fruitType { get; set; }
```

### Property Value

string

## quantity

```
public int quantity { get; set; }
```

### Property Value

int

## secColor

```
public string secColor { get; set; }
```

### Property Value

string

## size

```
public string size { get; set; }
```

### Property Value

string

## types

```
public string types { get; set; }
```

## Property Value

string

# Class TargetRelativeCalls

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class TargetRelativeCalls
```

**Inheritance**

object ← TargetRelativeCalls

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### relativeCalls

```
public RelativeCalls relativeCalls { get; set; }
```

#### Property Value

RelativeCalls

### source

```
public Pure source { get; set; }
```

#### Property Value

Pure

### target

```
public TargetObj target { get; set; }
```

## Property Value

TargetObj

# Class VariableInjection

Namespace: CJsonTests.models

Assembly: CJsonTests.dll

```
public class VariableInjection
```

**Inheritance**

object ← VariableInjection

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Properties

### jsonInjection

```
public string jsonInjection { get; set; }
```

#### Property Value

string

### target

```
public TargetObj target { get; set; }
```

#### Property Value

TargetObj